ELIOT: A Hardware Focused Experiment Builder

Tom Hilgers[1], Leander Schweiger[2], Steffen Müller[3], and Hans-Peter Heim[2]

[1]RWTH Aachen University, Germany
tom.hilgers@rwth-aachen.de
[2]Faculty of Mechanical Engineering, Institute of Materials Engineering – Plastics Technology (IfW), University of Kassel, Germany
{L.Schweiger, heim}@uni-kassel.de
[3]Faculty of Human Sciences, Institute of Psychology, University of Kassel, Germany
s.mueller@uni-kassel.de

Abstract

We present the ELIOT Experiment Builder (ELIOT), an open-source experiment builder that specializes in the integration of external hardware units into standardized laboratory experiments. ELIOT allows for various study designs that are customizable using HTML, CSS, and JSON. As an application for modularized JavaScript, it ensures flexible, robust, and comprehensive integration as well as synchronization of external hardware, by building on popular software frameworks. Studies and hardware code are packaged so that they can be shared with and adapted by other researchers, facilitating replication and standardization between laboratories. This paper serves as a tutorial to ELIOT. We provide an overview of the functionality, define terminology within ELIOT and explain the software's internal architecture. Central to data representation and collection in ELIOT are JSON objects. We therefore include a short introduction to JSON for behavioral researchers who are unfamiliar with the data format. The software code is provided under an open-source license and is available along with further documentation, ready-to-use installers, and support channels at https://github.com/eliot-org/eliot-experiment-builder.

Keywords: Psychology, Software, Experiment, Open-source, Hardware, Hardware integration, JSON, JavaScript

1. Introduction

Advances in open and accessible technology, like the Arduino Revolution (Kushner, 2011), Maker Movement (Dougherty, 2012), and hackathons (Pe-Than & Herbsleb, 2019), promise to facilitate laboratory experiments (D'Ausilio, 2012; Oellermann et al., 2022) by making them more flexible, cost-effective, and replicable. The lower economic and technical barriers to entry can enable laboratories with less funding and staffing to conduct more intricate experiments (Chagas, 2018). The used hardware is no longer specifically designed for one application but has a generic, i.e. more flexible, orientation. This makes it easier to share code between different laboratories, which can speed up a research process. In addition, results become more replicable, as it becomes easier and more affordable to acquire the same or equivalent hardware.

The opportunities that are offered by generic open-source hardware for laboratory-based behavioral experiments are not yet fully utilized by existing software or require considerable programming effort, and thus in-depth knowledge, by the user. Especially more complex experiments with interlocking and interdependent interactions pose a major challenge for inexperienced users, like laboratory or student assistants. The development and integration process in current experimental software is often intricate, challenging, and susceptible to errors. Furthermore, code for hardware interaction is often only developed for a specific experiment, which means that the reusability and flexibility of generic hardware is not fully leveraged, impeding standardization and replicability.

The primary goal of ELIOT is to be an application built on a modern software stack that makes it easier to leverage the advantages of generic hardware in a study design by taking the form of an open-source experiment builder that is comparable to other software such as Psychopy (Peirce et al., 2019), OpenSesame (Mathôt et al., 2012), and lab.js (Henninger et al., 2021). The Graphical User Interface (GUI) was developed to utilize the flexibility of web technology (HTML, CSS, and JavaScript) and has a simple, streamlined design so that the user can concentrate on the experiment process and the management of external hardware units. The latter is modularized and further standardized by predefined code norms that improve its reusability without negatively impacting customizability or extensibility. Designed to be flexible and easily extensible, the internal software architecture enables a quick development and prototyping process, while improving maintainability by building on popular software frameworks, including NodeJS[1], the npm package manager[2], and Electron[3]. We release the software under an open-source license to allow researchers to freely create, share, and reuse studies and code. We thus provide the research community with a flexible software for hardware-focused laboratory studies that meets the requirements of replicable scientific experiments.

We expand on existing work in a domain that has received less attention so far by combining and extending previously separate approaches. The development focus in recent years has been primarily on expanding the scope of experimental design, improving timing performance, and harnessing the potential of web browsers. Among primarily

[1]https://nodejs.org/
[2]https://www.npmjs.com/
[3]https://www.electronjs.org/

laboratory-based experimental software, Psychopy and OpenSesame are two popular solutions that provide extensive functionality via a comprehensive GUI with excellent timing performance (Bridges et al., 2020) and straightforward extendability via plugins. They also offer Python-based support for external hardware, with study design and code being intrinsically linked, meaning that the code for both is written in the same files. In contrast, ELIOT applies a modular, component-based approach, following software engineering practices (Pressman, 2005; Sommerville, 2011), to improve code robustness, readability, and reusability. lab.js is a recent browser-based software and is the culmination of a development history that includes jsPsych (De Leeuw, 2015), QRTEngine (Barnhoorn et al., 2015) and Psytoolkit (Stoet, 2017) as preceeding developments. All of these use JavaScript primarily to enable execution in the browser. JavaScript offers flexibility and customizability in experiment designs, because it allows easy access to and editing of HTML and CSS, which are the core building blocks of frontend web page design. Nonetheless, these tools are confined to the browser and can only interact with browser supported hardware, such as microphones or webcams.

ELIOT facilitates the support and leveraging of generic hardware in lab-based settings by using JavaScript in the form of NodeJS and JavaScript Object Notation (JSON) (ISO/IEC, 2017). JSON is a lightweight, standardized data format that is widely used for data exchange in web applications (Safris, 2019), it allows for simple exchange of data between different systems and applications. Our efforts included a simplification and standardization of the programming and study design process. Code is no longer written as part of a study, i.e. in the same files, but rather in separate components, which must follow predefined code interface characteristics. This is an important principle in software engineering (Pressman, 2005; Sommerville, 2011) as it separates different contextual domains, which has the effect of guiding the programmer's efforts in an orderly process, improving code readability, reusability between studies, and reducing error-proneness. For example, an error in an individal component will no longer interrupt the entire execution of an experiment, but only that of the specific component. Further, the separation is beneficial for new and inexperienced users, as the expertise required to modify the single JSON file that defines an entire experiment in ELIOT can be quickly acquired. Pre-developed hardware codes, like those published in the github repository, can be easily imported and used without coding abilities. Ideally, a collection of such codes, made for other studies and addressing new hardware, accumulates over time. ELIOT also includes integrated data management and merging, so that exporting data to external spreadsheet software is not a necessity. Together with the streamlined GUI, our software simplifies the process of creating and running a complex hardware-intensive experiment and managing the resulting data.

ELIOT was created because requirements for our own research projects were either insufficiently fulfilled by existing software or proved to be difficult to implement. Despite initially being developed as a specialized in-house project, we believe that our software and its design decisions will be of interest for the research community and therefore want to contribute by publishing the software under an open-source license in a public repository along with ready-to-use installers for Windows systems. In this way, it is possible to use the software freely and also customize it according to individual needs.

In addition, studies and hardware code can be freely shared, reused and adapted. It is important to note that, due to its development history, using ELIOT can require some technical know-how. This includes, in particular, knowledge of JSON, and, if required, JavaScript and NodeJS for hardware code development. In this paper, we will first describe the core concepts and terminology of ELIOT. We will then describe the user interface, followed by an examination of the software architecture. This is followed by an example experiment for which the software is particularly well suited, and concluding with a discussion.

2. General Concepts

This section of the paper provides an introduction to the underlying concepts that distinguish ELIOT from other software. The separation of study design and hardware code is discussed first. This is followed by a description of the survey file as a representation of the study design and an explanation of JSON.

2.1. Separation of Study and Code

A key feature of ELIOT is the separation of the study design from the code intended for hardware. This is done in a modular and component-based format by creating a separate piece of code, called script, for each external device. ELIOT as a software does not provide direct built-in hardware support but acts as an interface with predefined standards for the scripts, which are standard JavaScript that can make use of any NodeJS function and npm plugin. This means that a script can theoretically support any hardware device that can be connected to a PC in any way and for which adequate documentation exists. There are currently hardware script implementations that cover communication with an UR3 robotic arm via a local network connection and with Arduino-controlled scales, EEG trigger boxes, and LED lights via a serial interface. These scripts were developed for our own research, which integrates them into a complex and interlocking study process.

A script consists of at least one JavaScript code file and a JSON package file. The package file contains a configuration that describes how to handle the application contained in the code file. The format is the same as in any npm project[4]. Most importantly, the package file tells ELIOT where exactly the code file to import is located and what package dependencies it has. These dependencies are regular npm packages that will be installed by ELIOT upon import and enable all hardware interactions since standard JavaScript does not provide that functionality. For example, the plugin SerialPort[5] allows controlling devices on the USB ports of a PC. The conventions that the main code file must follow are that it must be able to accept incoming data from the application, send data back to it, and be able to interact with the hardware it was designed for. A more detailed explanation is given in the Hardware section and example implementations can be found in the github repository.

[4]See the npm documentation for further details at https://docs.npmjs.com/
[5]https://serialport.io/

2.2. Survey File

Building a study in ELIOT is done separately from the GUI by creating a single survey file (see Listing 1). It is in the JSON format, which we will describe in the next section, and is described by detailed schemata that are available in the ELIOT repository. There, we also provide example survey files, of which one describes a common Stroop task (MacLeod, 1991; Stroop, 1935) in the form of a color-word interference test. A survey file contains modules, parts, and the survey definition. Modules and parts model the overall structure of the survey and have the accompanying requirement that each question must be part of a module. Each module is divided into several parts that allow for finer control of the survey flow. For example, one module might be a study section on Person-Related Information (PRI), and questions about the subject's demographics would be found in this *"PRI"* module and the *"Demographics"* part. In a study where only the subject's demographic data is needed, only the *"Demographics"* part would be selected, so that other parts of the *"PRI"* module would be skipped. A simple analogy for modules and parts are the chapters and subchapters of a book, where the chapters are the modules, and the subchapters are the parts.

The survey definition consists of a set of JSON objects, each of which represents a single page that can be displayed in an experiment run. For each of these pages the `module`, `part`, `type`, and `content` attributes must be defined. These attributes define the characteristics of the presented page, i.e., which and how stimuli are presented. For each page type, which will be described in detail in the Survey Window section, different content options are available. For example, all page types contain the `text` attribute, which defines a text that will be displayed on the page, but not all have an `image` attribute which contains the name or path of an image to be displayed. There are many more type-attribute combinations, all of which can be found in the schemata. For each page JSON object, there is an optional `hardware` attribute that defines the interaction of this page with the connected hardware, as can also be seen in Listing 1. The value of the hardware attribute is again a JSON object with multiple attributes to choose from. One example is `OnPageLoad`, which contains one or more commands that are executed immediately when the page is loaded, each of which instructs a device to perform a particular action. A device, which corresponds to the device name that can be set on the hardware page of the program, has to be specified for every command. A command is executed, if the target device exists and its script has defined the given command. It is also possible to bind specific actions, like setting an answer value, to a keypress using the key attribute.

2.3. JSON

ELIOT utilizes the JSON format, which is a human readable, text-based format for representing structured data. It is commonly used for storing and transmitting data in web applications (Safris, 2019). JSON follows a simple structure and many programming languages and environments can read and generate JSON. A JSON object consists of attribute-value pairs, where the values can be strings, numbers, arrays (sets), and JSON

objects, among others. The following example shows a single JSON object, represented by the curly brackets, with its attributes and their values:

```
{"firstname": "Tom", "lastname": "Hilgers", "University": "RWTH Aachen",
    ↪ "aSetOfNumbers": [1, 2, 3, 4, 5], "anObject": {"hello": "World"}}
```

The attribute values can also be JSON objects themselves, as seen in the `anObject` attribute. Therefore, deeply nested, but still readable data structures are possible. Because of its versatility and integration into JavaScript, JSON was chosen for all data storage and representations in ELIOT. Conversion from JSON to other formats, such as CSV or XML, is also possible. JSON schema (Wright et al., 2022) (Wright et al., 2017) was used, to provide users with detailed instructions on how to create survey files. It follows the same syntax as JSON and contains keywords that specify the structure and type of the JSON object to be created. It defines which attributes can be used, where they can be used, and what type of value they can take.

3. Graphical User Interface

To increase flexibility for the study conductor, we decided to split our interface into two separate windows. The first one is called admin window, while the second is called survey window and shown to the test subject. This separation enables the conductor to monitor the course of the experiment, i.e. to confirm manual steps, or to pause or abort the experiment, while the test subject answers the survey, even if they are spatially separated, e.g. in a different room. The survey window is only open when a study is being conducted.

3.1. Admin Window

Several pages are available in the admin window, each corresponding to a function that might be useful for a user. We will now describe these pages and their functions in the order in which they appear in the program.

3.1.1. Survey

After starting ELIOT, the user is presented with the survey page of the admin window, i.e., the main page for designing a study, see Figure 1 for reference. The first step in building an experiment is to select a survey file, which must first be imported on the settings page. Then the user can choose which modules and parts they want to present by selecting and adding at least one. It is also possible to add all available modules with one click. The order of the added modules can be changed via drag and drop. It is possible to attach multiple stimulus objects, which will be explained in a later section, to each added module, resulting in a repetition for each stimulus object. A module can also be repeated multiple times independently of the attached stimulus objects. This means that repeating a module with two stimulus objects twice results in four repetitions. It is also possible to randomize the order of the stimulus objects and of the parts of a

module using the Fisher-Yates-Shuffle (Fisher & Yates, 1948). The name of the study, experimenter and research assistant can also be specified. The input in these fields will be saved in the results along with the answers to the study. It is also possible to choose a specific screen on which to open the study. When all options are set, a study can be started whereby the survey window will open on the specified screen for the participant to view it.

3.1.2. Subjects

The program keeps a record of the participants of a study, also referred to as subjects within ELIOT. These can be created either manually or through their participation in a study. The subject page gives an overview of the internally stored subjects and their data attributes (see Figure 2). Each subject has at least one attribute which is a unique identifier in the form of a six-digit code that can be used to identify subjects in future surveys. Given the nature of the JSON storage format, any number and type of attribute can be added, e.g., sociodemographic variables. Subjects and their attributes can be manually created, or automatically in an experiment run by adding the `definesSubject` attribute to a question in the survey file. This is illustrated in the example in Listing 1. The value set in this attribute defines which attribute-value pair of a subject will be created and set by the answer to this question. At the end of a survey, all these pairs are collected and stored together with a newly generated code as a new subject. In addition, it is possible to use the attributes of the participating and identified subject in a survey. This means similar to values of a stimulus object's attribute, the value of a subject's attribute can be injected into a page. For example, the height of a subject can be used in a question text. This requires that the subject has previously entered their code on a page provided for this purpose at the start of the survey. It is possible to export all subject data in a JSON file.

3.1.3. Stimulus Objects

Stimulus objects represent the study-specific stimulus material that is presented to the test subject either in physical or virtual form, as images, videos, sound recordings or any combination of the above. Stimulus objects are tangible things that a study can revolve around and are not to be confused with JSON objects. Unlike a subject, a stimulus object cannot be created through an experiment run because it defines part of what is presented in these experiments. The attributes of stimulus objects can be used within an experiment, for example images (image paths) can be stored and then displayed in the survey. As with subjects, they are internally stored in the JSON format and it is possible to export all data in a JSON file.

3.1.4. Measurements

This page provides an overview of the responses collected during surveys (referred to as measurements within ELIOT); an example is given in Figure 3. For each measurement, the data is stored separately. A stored measurement consists of the subject code, the

date, the duration of the study, an id, and a set of answers. Each of these answers consists of the question text, the question type, the given answer, the module, the part, the time (in seconds, with a nanosecond resolution) that the page was displayed or that it took to react to a question, and the unique name of the question. Measurements can be exported as JSON and can optionally include the respective subject data.

3.1.5. Hardware

This page is central to ELIOT, as it allows the user to configure and interact with connected hardware (see Figure 4). A console is located in the upper part of the page, displaying all events and messages regarding the hardware. Following that is an overview of the loaded scripts. Every script has a name, a description, lists of available commands, possible events and connected devices and the possibility to connect a new device. It is also possible to manually send commands to devices to test their functionality. A more detailed explanation of hardware and scripts will follow in the Hardware section.

3.1.6. Settings

On the settings page, the last page in the admin window, the user has the option to make general settings. Here they can specify the location of the hardware scripts and import or remove survey files. It is also possible to specify the folder where images and other files that can be displayed in the surveys are located.

3.2. Survey Window

ELIOT offers many different ways to present or collect information from a test subject. This is done through different page types in the survey window, which we will now describe.

3.2.1. Presentation

Multiple page types are provided to present information. These include the page type `explanation`, `explanationPic`, `manual`, `welcome`, and `end`. The following list describes these page types:

- `explanation`: Any text or text in HTML format can be displayed in the center of the page.
- `explanationPic`: An extension of explanation where images and videos can also be displayed. Their file names must be specified in the survey file for this purpose. It is also possible to show the information and images in different configurations. For example, just the image/video in full screen or above/below a text.
- `manual`: Based on explanationPic, but continuing to the next page can only be done by the study conductor. For this purpose, after loading a manual page, a button appears in the admin window that the conductor can press to continue to

the next page. This step can also be done with hardware events (described in the Hardware section).

- `welcome`: Here not only text can be shown as in explanation, but the subject can also enter his possibly already existing code. This code can then be used to inject subject attributes into the pages.
- `end`: It is used to say goodbye to the subject and to inform them of their personal six-digit code.

3.2.2. Timer

A special type of presentation is the timer page, which consists of three phases. Each phase shows information only for a certain period of time after which the next phase starts. For each phase, the timer length, image and text can be set. The second phase is the main phase in which information can be presented, while both the first and third phases are intended as buffers to separate the main phase from the previous and next pages. For example, a test subject can be instructed in phase one to prepare for the interaction with a material sample. In phase two, they can then perform this interaction. Phase three can then be used to return to the starting position. The time limit in the phases ensures, that the whole process is executed in a previously defined time period.

3.2.3. Question

When data is to be collected, the question page is used. Here, a question is asked in the left-hand area of the screen, possibly with the help of images, and an input is made in the remaining area of the screen. It is important to note that when setting the `type` attribute of a page in the survey file to question, there will be another `type` option inside the content that has to be set, as seen in the example in Listing 1. This secondary type attribute sets the specific type of question that will be shown. There are a variety of options for this, which we describe below, with the abbreviations used in the program and the survey files enclosed in brackets. There is a standardized slider (slider) input, in a categorical or metric scale. The scaling is freely adjustable. A gradient from red to green can be added. Optionally, it is possible to display one or more sliders per page. Feedback from multiple sliders can be displayed on a page via a radar chart (polygonSlider) as shown in Figure 5. It is also possible to display the average slider value below the radar chart. Other standard formats included are checkboxes (checkbox), radio boxes (radio), continuous text input (text), yes/no selection (yes-no) and selection from a dropdown menu (select). A less common question format is a dual-axis rating (matrix) for two continuous variables in relation to a common matrix. This can for example be used in joint assessment of valence and arousal to capture the current emotional state. It is possible to extend this by another dimension by growing or shrinking a circle in the input area (matrix_circle). Figure 6 and Figure 7 show these two input forms.

3.2.4. Canvas

The canvas page has a very basic structure that gives users the opportunity to freely design it according to their needs. There are no given design elements, like the left-hand area of question pages, that might not be needed for a specific task. A canvas page can be used to present information, but also to question the survey subject. Our stroop example survey file uses the canvas page to only show colored words without any other design elements that might influence the subject in reacting to the word with a keypress. The JSON properties available are `name`, `style` and `html`. A user can use the `style` property to design the background, while the `html` property describes what will be displayed on top of the background.

4. Software Architecture

The Electron framework was chosen as the basis for the software architecture and user interface. It allows for the creation of a website as an ordinary program, together with all features of NodeJS. A website usually consists of HTML, CSS, and JavaScript, which together are responsible for presentation and functionality. With Electron, it is possible to display this combination locally, without an internet connection and without a dedicated web browser. Crucially, and unlike python-based templates, these local websites can be used in web-based settings without requiring major changes, providing significantly more opportunites for using a single design template. This was part of our considerations in choosing a framework because an initial objective in the development of ELIOT was the ability to easily port a study design developed for the laboratory to an online-based setting without noteworthy differences in presentation. Electron also offers the ability to use the advanced capabilities of NodeJS. It can, with its npm package manager, also import packages of bundled JavaScript code that implement features that would otherwise be difficult or time-consuming to develop. Our hardware integration is only made possible by these packages. They allow our JavaScript code to access the hardware interfaces of the computer, for example USB ports, and communicate with the hardware devices. Many packages are privately developed and open-source software. A program created with Electron consists of two parts or processes. The main process that works in the background and acts like a web server based on NodeJS, and the renderer process that displays the pages and is based on Chromium, the open-source version of Chrome. Programs created with Electron also offer the freedom of cross-platform functionality. However, modern websites rarely consist of pure HTML, CSS and JavaScript. Nowadays it is common to use a so-called web framework. These make it possible to implement many features, which normally require significant programming effort, easier and faster. For ELIOT we chose Vue.js[6] since it is open-source, easy to use, and because there is a sample integration in Electron that can be used as a starting point[7]. The development tools integrated in Chromium and thus also Electron, together with the low-level output

[6] https://vuejs.org/

[7] This sample integration is a privately maintained open-source project and can be found under https://github.com/SimulatedGREG/electron-vue.

of NodeJS, allow a comprehensive assessment of technical integrity and correctness by a developer or open-source contributor of ELIOT. Because of the integrated nature of these frameworks, apart from the initial download of the installer, an internet connection is not required for the installation and execution of ELIOT.

4.1. Hardware

The principle in our implementation is that a user creates a script to interact with a particular piece of hardware. A script resides in a folder as a code file, along with any other files that may be required and a mandatory package JSON file. This must be in the usual style of a npm package file and specify all dependencies of the script. A script can be loaded into the program via the settings page. As previously established, it must conform to some conventions. Most importantly, it must define which parameters are needed to establish a connection with a particular piece of hardware. This includes for example the COM port or the IP address. Furthermore, it must be defined which events can be sent from the scripts or the hardware to the ELIOT program and how data can be received from it. The events can be used in an experiment run to load the next page in the program by, for example, pressing a button in the real world. Lastly, the commands have to be defined. These are the opposite of events and are sent by the program to the hardware. For example, a command might be used to trigger an EEG trigger box. These commands can be used in survey files. To allow for more flexibility in the interaction between hardware and software, the attributes of the currently shown page are also sent to all hardware scripts. This includes the current page type, the content of the page, the current, and the next stimulus object.

5. Example Experiment

In this section we will apply the previously presented information about ELIOT by broadly outlining how an experiment with hardware integration can be created. This example will demonstrate the creation process and highlight the type of experiment that ELIOT is particularly suited for, because it involves interactions between different, interdependent hardware units. The example is inspired by previous research of Schweiger and Heim (2023) that prompted the development of ELIOT.

The goal of the experiment is to measure the response of a subject to haptic perception of a material sample, using an EEG and a survey. Several hardware units were required in order to perform this experiment automatically and with minimal intervention by the study conductor. The first unit was a trigger box, that sets time stamps in the EEG recording when the subject haptically interacts with the material. This was achieved by using a scale to measure the weight change caused by placing a hand on the material sample. For the visual presentation, the material was illuminated for a fixed period of time. Both the lighting and the scale are realized by two independent arduino setups. This experiment is meant to serve as an example; other setups, extensions, and processes are also possible. In a similar research setting, we further extended the setup with a robot arm that was controlled with ELIOT.

As described previously, to create an experiment as described above within ELIOT, the necessary hardware scripts are written before the actual creation of the experiment begins. For this purpose, a dedicated script is created for each device, covering its specific functionalities. The commands and events that will be used in the survey file are also defined. For example, the scale should send a signal to the trigger box that sets a timestamp in the EEG recording, specifically in response to the subject's interaction with the material sample, rather than to random measurement fluctuations. Therefore, the command `enable` is defined in the script of the scale, which starts the weight measurement and ensures that, after a certain change in weight, an event called interaction is sent. This event is then received by all other scripts and causes the script of the trigger box to send the timestamp signal. Such a chain of events can take any length. Once the scripts have been created, they are then loaded on the hardware page of the program and a connection to the devices can be established. Messages and errors are displayed in the integrated console.

The next step in the setup of this experiment is the creation of the survey file. For this purpose, the desired modules, parts and pages to be displayed are defined. The survey can consist of any questions and explanations, which can be modified freely according to the available options. Stimulus object data might also be used to alter the content of pages, for example, to load an image of the object. The stimulus interaction is linked to a timer page, which ensures that the subject interacts with the sample for a predefined time interval. The relevant hardware commands are also used here, in this case `enable` for the scale and `on` to activate the lighting. For this purpose, the device names defined on the hardware page in the program must also be entered. After the survey file has been created, it can be loaded into the program via the settings page. Before the experiment can be started, the subject and the stimulus objects, corresponding to the investigated materials, must be created and selected. At this point, the experiment can be started and will run automatically without requiring further input from the conductor. The data collected during the survey is saved afterwards and can be viewed on the measurement page and exported as JSON.

6. Discussion

ELIOT is a novel experiment builder designed for laboratory-based research with a comprehensive support for external hardware devices. It applies important software engineering practices, like modularization, to experiment design and hardware code creation. Unlike existing software solutions, experiment design and hardware integration are no longer created jointly and interdependently. Instead, they are developed independently, with the hardware integration being incorporated as discrete components into the software framework. ELIOT also capitalizes on web technologies in multiple ways. First, JavaScript in the form of NodeJS and Electron enables a quick development process, e.g. with the npm package manager. Second, HTML and CSS to ease the design process. And third, JSON to store experiment definitions in single, human-readable, and well-defined files.The definition of an experiment is stored in a single, well-defined JSON file. In

addition, the software manages the collected data internally. Through these features, the capabilities of modern web technology are applied to laboratory-based, hardware-focused software for the first time.

The features of ELIOT offer several potential benefits for the research community. Since studies are directly provided as a single JSON file and follow a standardized format, they can be easily used and understood by other researchers, requiring only a basic level of technical know-how. Likewise, the independently created and modularized code is more universally usable, which not only enhances its usefulness for future studies but also for other researchers when shared. In addition, modularized code is also generally easier to understand, more robust, and easier to maintain. Since the same conventions apply to all hardware scripts, a repository of scripts for a wide variety of hardware devices, that go beyond the current implementations, may accumulate over time. This would make it easier for new users to get started with their own research, because they could import these pre-developed scripts and use them without needing coding abilities. Ultimately, these features hold the promise of improving the reusability, shareability, and replicability of experimental research in general.

Despite these important advantages over established software, ELIOT also has several limitations that must be taken into account. First, it is not possible to create a survey directly in the GUI. Rather, survey creation is performed manually in a JSON file, following the specific rules that underlie survey files. This process might be time-consuming and unintuitive for inexperienced users. Popular experimental softwares in contrast, both in the lab and in the browser, possess comprehensive builder interfaces. Second, the potential of web technology to offer a more flexible frontend is not fully realized in ELIOT. In contrast, lab.js, for example, offers a fully customizable study interface, removing the need for survey creators to rely on templates and other pre-defined designs. ELIOT offers this feature only to a limited extent through the predefined options for the question pages. Third, the potential study designs and sequences are also limited by the lack of branching and dependencies. As a result, questions cannot depend on the answers to previous questions, only some of the content can be changed dynamically by subject attributes and hardware interaction. These limitations are largely due to the early stage of development and are not insurmountable. As such, they should be prioritized in future development efforts. We were unable to investigate the timing accuracy, the underlying NodeJS functions are accurate in theory but may be susceptible to inaccuracies in applied settings, e.g. due to monitor refresh rates or hardware limitations.

Through the development of ELIOT, we have endeavoured to create a new software for conducting hardware-focused and laboratory-based behavioral experiments based on a modern software stack. We applied essential software engineering practices to our development process and design decisions. We have commited to the principles of open-source software by freely providing the software and making the resulting studies and codes available as easily shareable resources. We hope that ELIOT will enable researchers to create robust, laboratory-based, and hardware-dependent experiments.

Acknowledgements

Initial versions of this software were developed as part of the employment of Tom Hilgers and Leander Schweiger at imat-uve gmbh. We would like to thank Hans-Willi Müller for his permission to make the software open-source, thus making the creation of this paper possible.
This Project is supported by the Federal Ministry for Economic Affairs and Climate Action (BMWK) on the basis of a decision by the German Bundestag.

Author contributions

Tom Hilgers served as lead for conceptualization, software, and writing - original draft. Leander Schweiger served in a supporting role for conceptualization and writing - original draft. Steffen Müller served in a supporting role for conceptualization and writing - original draft. Hans-Peter Heim served in a supporting role for writing - review & editing. Tom Hilgers, Leander Schweiger, and Steffen Müller contributed to writing – review & editing equally.

References

Barnhoorn, J. S., Haasnoot, E., Bocanegra, B. R., & van Steenbergen, H. (2015). QRTEngine: An easy solution for running online reaction time experiments using Qualtrics. *Behavior research methods*, *47*, 918–929. https://doi.org/10.3758/s13428-014-0530-7

Bridges, D., Pitiot, A., MacAskill, M. R., & Peirce, J. W. (2020). The timing mega-study: Comparing a range of experiment generators, both lab-based and online. *PeerJ*, *8*, e9414. https://doi.org/10.7717/peerj.9414

Chagas, A. M. (2018). Haves and have nots must find a better way: The case for open scientific hardware. *PLoS biology*, *16*(9), e3000014. https://doi.org/10.1371/journal.pbio.3000014

D'Ausilio, A. (2012). Arduino: A low-cost multipurpose lab equipment. *Behavior research methods*, *44*, 305–313. https://doi.org/10.3758/s13428-011-0163-z

De Leeuw, J. R. (2015). jsPsych: A JavaScript library for creating behavioral experiments in a Web browser. *Behavior research methods*, *47*, 1–12. https://doi.org/10.3758/s13428-014-0458-y

Dougherty, D. (2012). The maker movement. *Innovations: Technology, Governance, Globalization*, *7*, 11–14. https://doi.org/10.1162/INOV_a_00135

Fisher, R. A., & Yates, F. (1948). *Statistical tables for biological, agricultural and medical research (3rd ed.)* Oliver; Boyd.

Henninger, F., Shevchenko, Y., Mertens, U. K., Kieslich, P. J., & Hilbig, B. E. (2021). lab. js: A free, open, online study builder. *Behavior Research Methods*, 1–18. https://doi.org/10.3758/s13428-019-01283-5

ISO/IEC. (2017, November). *The JSON data interchange syntax* (Standard). International Organization for Standardization. Geneva, CH. https://www.iso.org/standard/71616.html

Kushner, D. (2011). The making of Arduino. *IEEE spectrum*, *26*, 1–7.

MacLeod, C. M. (1991). Half a century of research on the Stroop effect: an integrative review. *Psychological bulletin*, *109*(2), 163. https://doi.org/10.1037/0033-2909.109.2.163

Mathôt, S., Schreij, D., & Theeuwes, J. (2012). OpenSesame: An open-source, graphical experiment builder for the social sciences. *Behavior research methods*, *44*, 314–324. https://doi.org/10.3758/s13428-011-0168-7

Oellermann, M., Jolles, J. W., Ortiz, D., Seabra, R., Wenzel, T., Wilson, H., & Tanner, R. L. (2022). Open hardware in science: The benefits of open electronics. *Integrative and comparative biology*, *62*(4), 1061–1075. https://doi.org/10.1093/icb/icac043

Peirce, J., Gray, J. R., Simpson, S., MacAskill, M., Höchenberger, R., Sogo, H., Kastman, E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior research methods*, *51*, 195–203. https://doi.org/10.3758/s13428-018-01193-y

Pe-Than, E. P. P., & Herbsleb, J. D. (2019). Understanding hackathons for science: Collaboration, affordances, and outcomes. *Information in Contemporary Society: 14th International Conference, iConference 2019, Washington, DC, USA, March 31–April 3, 2019, Proceedings 14*, 27–37. https://doi.org/10.1007/978-3-030-15742-5_3

Pressman, R. S. (2005). *Software Engineering: A practitioner's approach (6th ed.)* McGraw-Hill.

Safris, S. (2019). A Deep Look at JSON vs. XML, Part 1: The History of Each Standard [Accessed December 3, 2023]. https://www.toptal.com/web/json-vs-xml-part-1

Schweiger, L., & Heim, H.-P. (2023). Haptic Polymer Testing: A Method for Evidence-Based Material Perception Measurement. *Manuscript submitted for publication.*

Sommerville, I. (2011). *Software Engineering (9th ed.)* Addison-Wesley.

Stoet, G. (2017). PsyToolkit: A novel web-based method for running online questionnaires and reaction-time experiments. *Teaching of Psychology*, *44*, 24–31. https://doi.org/10.1177/0098628316677643

Stroop, J. R. (1935). Studies of interference in serial verbal reactions. *Journal of experimental psychology*, *18*(6), 643. https://doi.org/10.1037/h0054651

Wright, A., Andrews, H., Hutton, B., & Dennis, G. (2022, June). *JSON Schema: A Media Type for Describing JSON Documents* (Internet-Draft No. draft-bhutton-json-schema-01) (Work in Progress). Internet Engineering Task Force. Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-bhutton-json-schema/01/

A. JSON Listing

Listing 1: Example survey file

```
{
    "name": "Example Surveyfile",
    "modules": [
        {
            "id": "Welcome",
            "label": "Welcome",
            "children": [
                {
                    "id": "Welcome Page",
                    "label": "Welcome Page"
                }
            ]
        },
        {
            "id": "PRI",
            "label": "PRI",
            "children":[
                {
                    "id": "Demographics",
                    "label": "Demographics"
                },
                {
                    "id": "Preferences",
                    "label": " Preferences"
                }
            ]
        },
        {
            "id": "Ending",
            "label": "Ending",
            "children": [
                {
                    "id": "Ending",
                    "label": "Ending"
                }
            ]
        }
    ],
    "survey":[
        {
            "module": "Welcome",
            "part": "Welcome Page",
            "type": "welcome",
            "content": {
                "text": "Welcome to this Experiment <br> Do you want to start?"
            }
        },
        {
            "module": "PRI",
```

```
            "part": "Demographics",
            "type": "explanationPic",
            "content": {
                "text": "This is the current objects imagename: {{obj.img}}",
                "img": "{{obj.img}}",
                "imagePosition": "below"}
            },
        {
            "module": "PRI",
            "part": "Preferences",
            "type": "question",
            "content": {
                "definesSubject": ["checkboxAnswer"],
                "text": "The Answer to this question defines an attribute of the subject",
                "name": "checkboxQuestion",
                "type": "checkbox",
                "options": ["1", "2", "3", "4", "5"]
            },
            "hardware": {
                "onPageLoad": [
                    {
                        "device": "lights",
                        "command": {
                            "name": "turnOn"
                        }
                    }
                ]
            }
        },
        {
            "module": "Ending",
            "part": "Ending",
            "type": "end",
            "content": {
                "text": "Goodbye"
            }
        }
    ]
  }
```

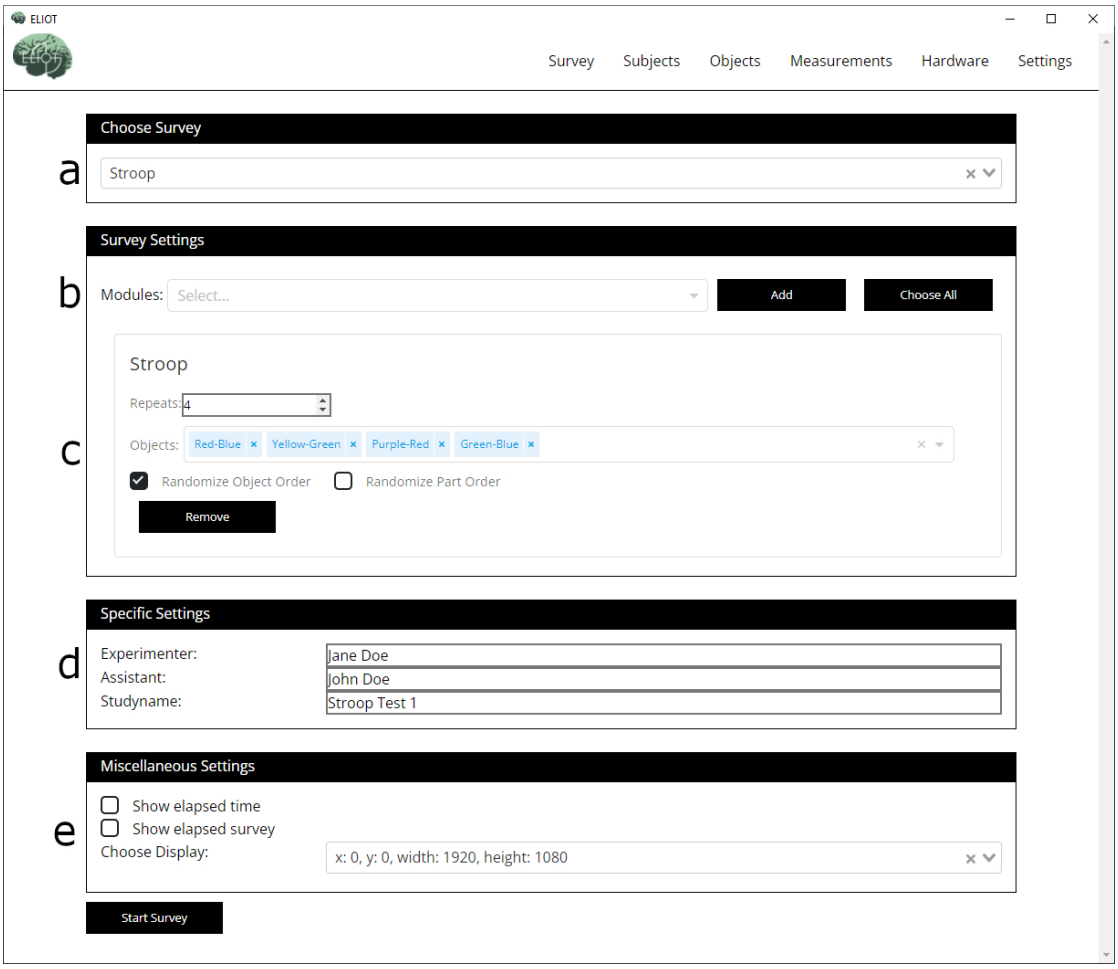

Figure 1: The ELIOT interface after selecting everything needed to run a survey. **a** Survey file selection. **b** Module selection. **c** Module settings (Repeats, stimulus object selection, randomization setting, drag & drop of modules). **d** Settings specific to this study. **e** Miscellaneous settings.

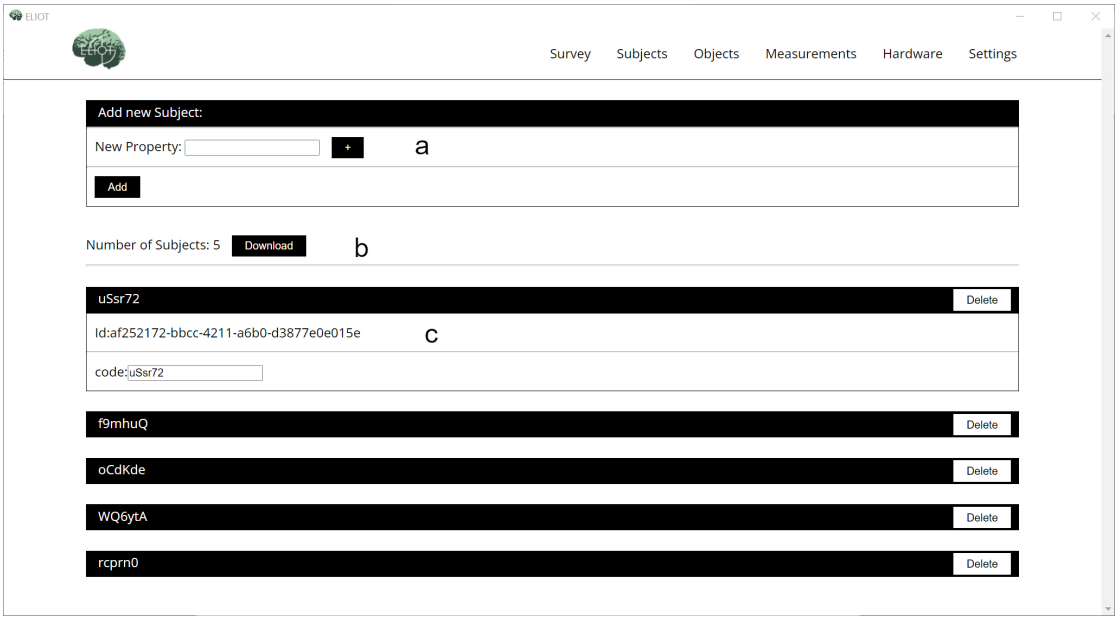

Figure 2: The subject page in the admin window showing 5 subjects. **a** Add a new object by adding properties and setting their value. **b** The list of stored subjects. **c** The detailed view of a subject with his attributes.

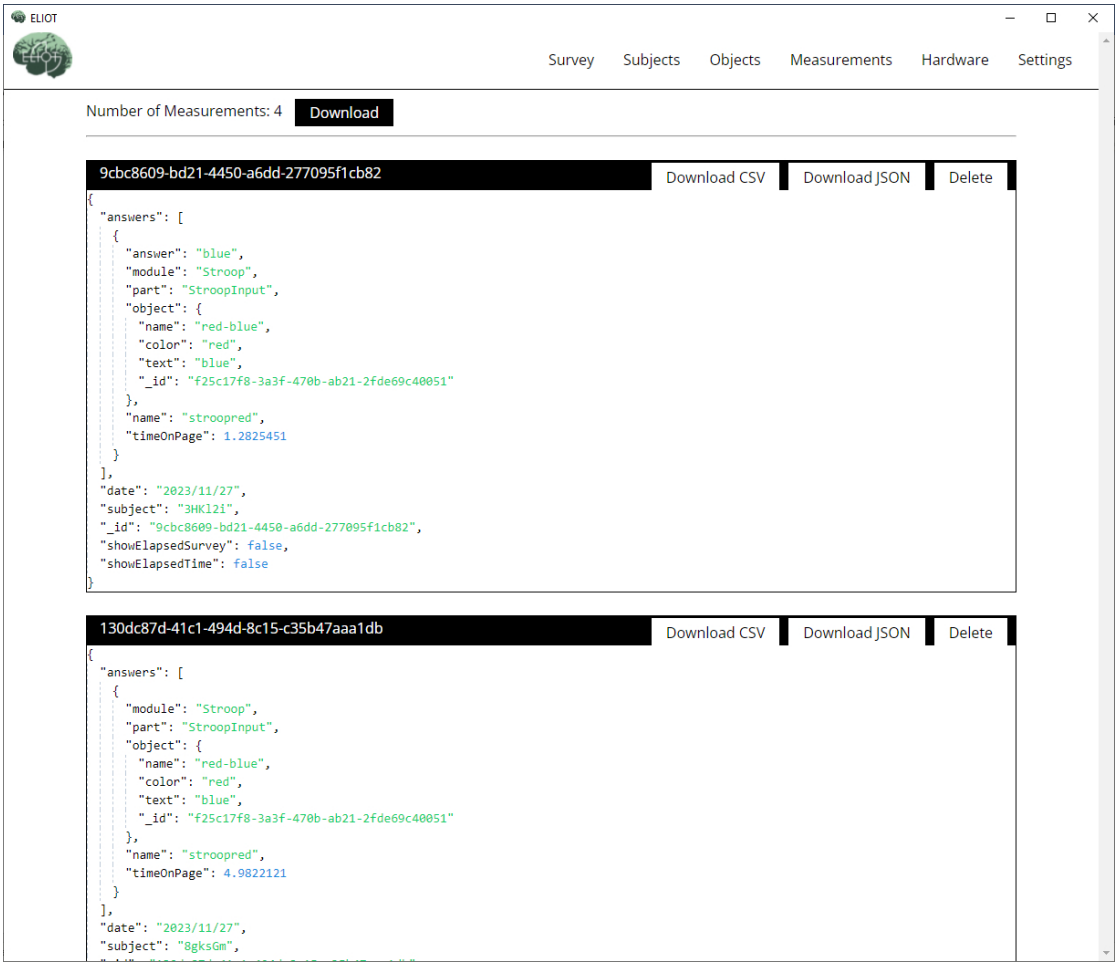

Figure 3: The details of two measurements are expanded in the figure.

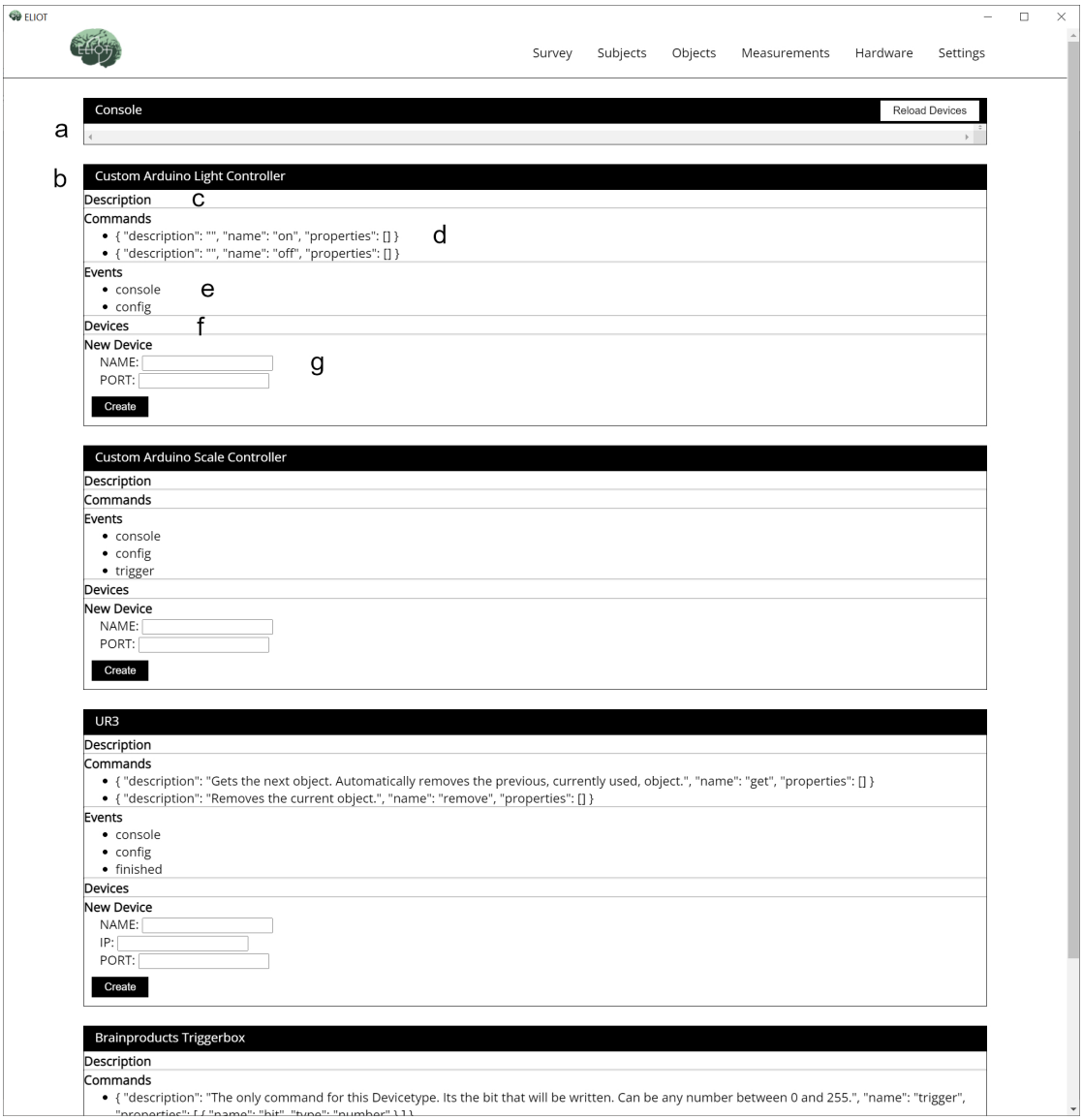

Figure 4: The hardware page when multiple scripts are loaded. **a** The console for event output. **b** A single scripts details. **c** Description of the script. **d** Available commands for this script. **e** Possible output events. **f** Currently connected devices. **g** Options for connecting a new device.

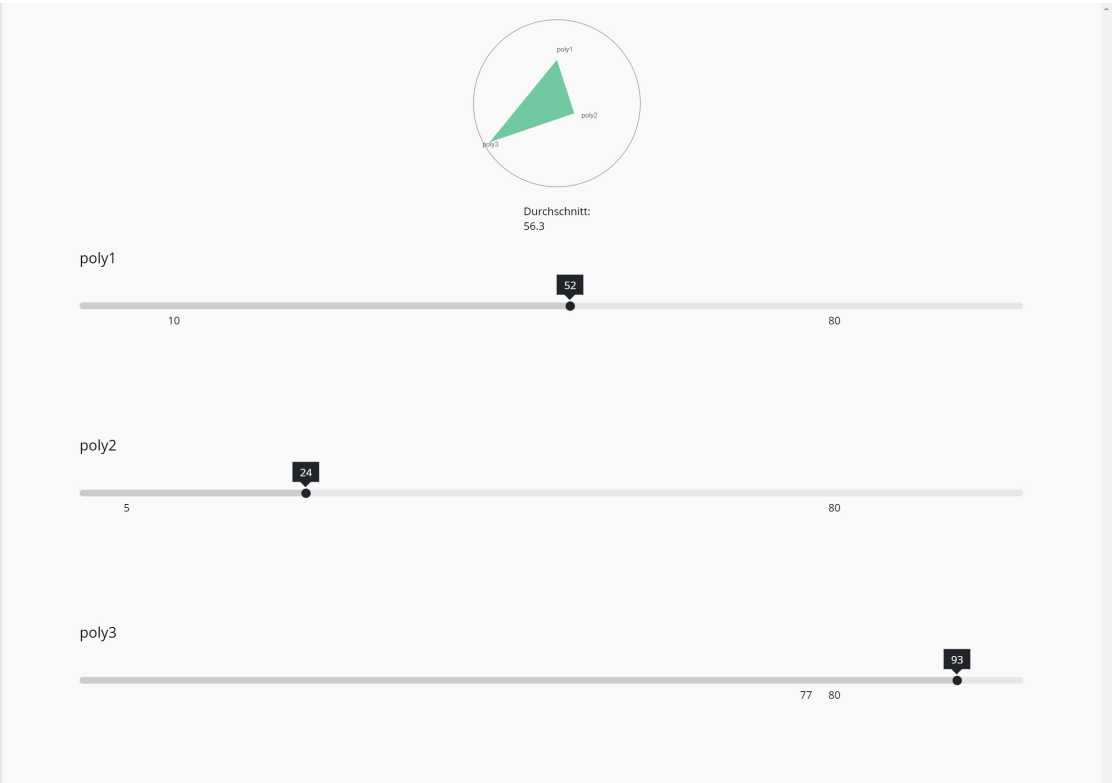
poly1
poly2
poly3
Durchschnitt:
56.3
poly1
52
10
80
poly2
24
5
80
poly3
93
77
80

Figure 5

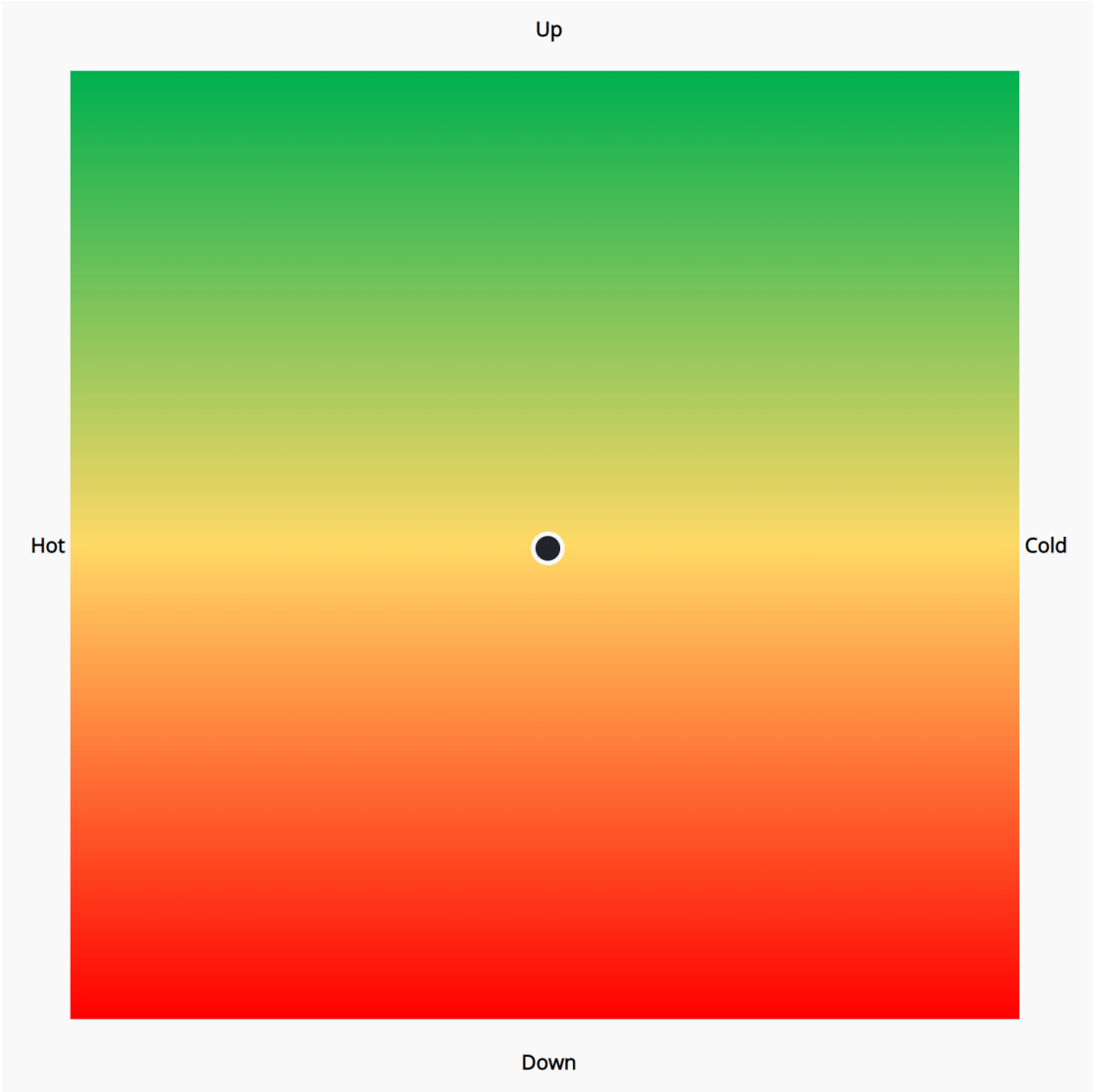

Figure 6

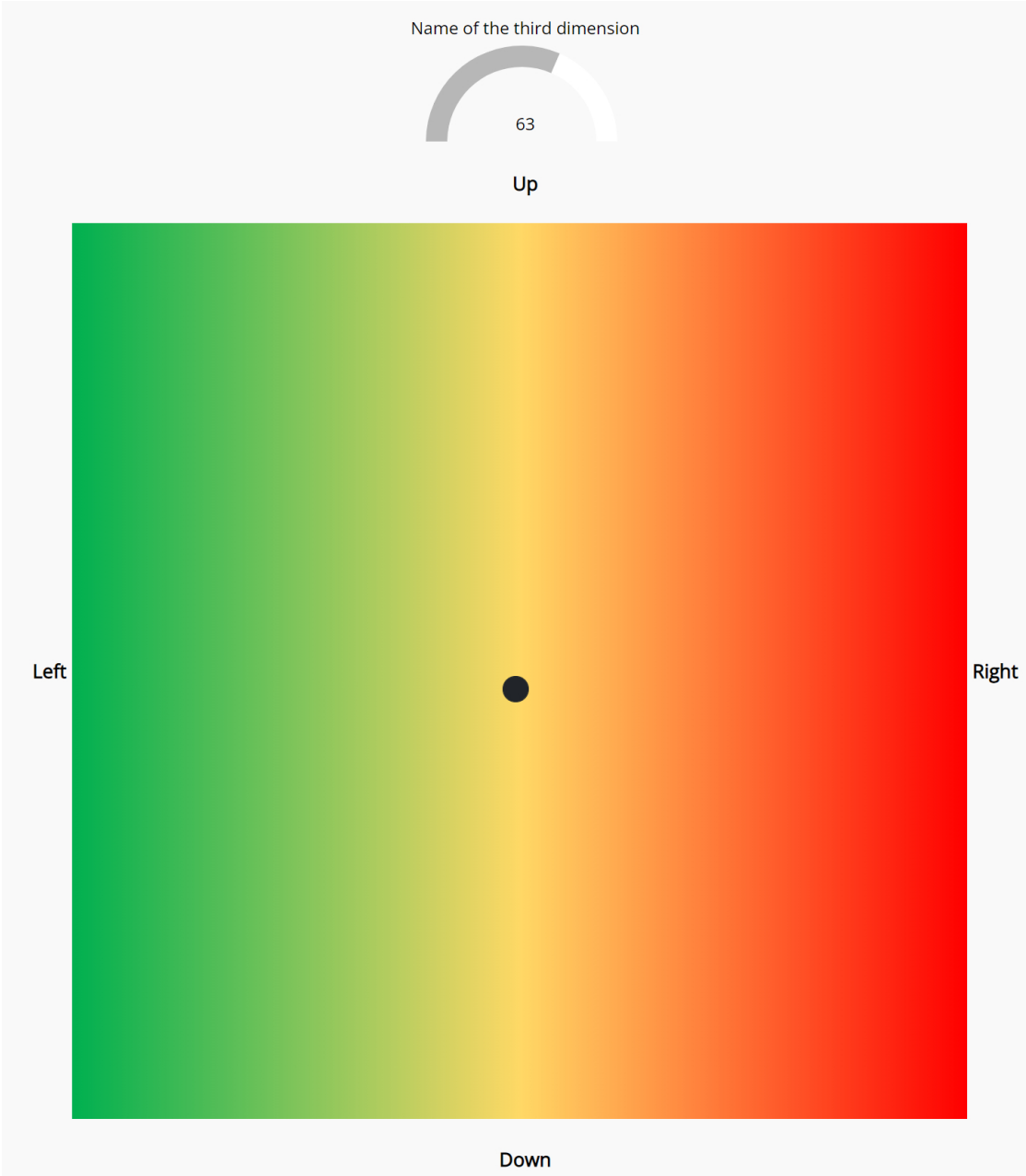

Figure 7